SYSTEM DESIGN DOCUMENT

Bootstrap Server for Distributed Real-Time Media Synchronization

Project architecture • Phase 1 / Initial Design • Expanded and presentation-ready version

CORE ARCHITECTURAL PRINCIPLE
The bootstrap server establishes the initial network; the peer layer performs synchronization.

1. System Overview

The proposed system is a decentralized, leaderless protocol and SDK for real-time media synchronization. The long-term objective is to minimize server dependency and move synchronization state and decisions into the peer network. The initial design introduces a small bootstrap/signaling server because browsers need assistance with session discovery and WebRTC connection establishment.

Client / Browser

Runs the media application and client SDK. Participates in WebRTC and synchronization.

Bootstrap Server

Creates/discovers sessions and temporarily exchanges WebRTC signaling information.

Peer Network

Carries synchronization messages and will eventually manage distributed state and timing.

2. System Design Basics Applied to This Project

Requirement: Multiple browser users need to watch synchronized media in real time.

Constraint: The target architecture should avoid a permanent centralized synchronization authority.

Approach: Use a lightweight bootstrap server for discovery/signaling and WebRTC for peer communication.

Target property: Synchronization state and decisions should eventually be handled by peers rather than by the bootstrap server.

System boundary

Inside the Phase 1 boundary: room creation, room joining, temporary peer identity, peer discovery, WebRTC signaling, direct connection establishment and initial peer-message exchange. Outside the Phase 1 boundary: the complete leaderless synchronization algorithm, distributed clock, drift correction, convergence, partition handling and malicious-peer handling.

3. System Design Fundamentals

The design is easier to understand when the system is decomposed by responsibility.

3.1 Functional requirements

• A user can create a synchronization room.

• Another user can discover and join an existing room using its room identifier.

• The system can assign temporary peer/session identifiers.

• Participants can exchange the WebRTC signaling information required for connection establishment.

• Browsers can establish peer-to-peer connections where possible.

• Peers can exchange synchronization messages after the connection is established.

3.2 Non-functional / architectural goals

• **Low server dependency:** the server should not remain in the normal synchronization path.

• **Decentralization:** important synchronization logic belongs in the peer layer.

• **Extensibility:** the synchronization engine should remain independent of the server framework and transport.

• **Scalability investigation:** the final peer topology is intentionally left open for experiments.

• **Fault tolerance:** future layers must address disconnection, reconnection and network partitions.

3.3 Logical component decomposition

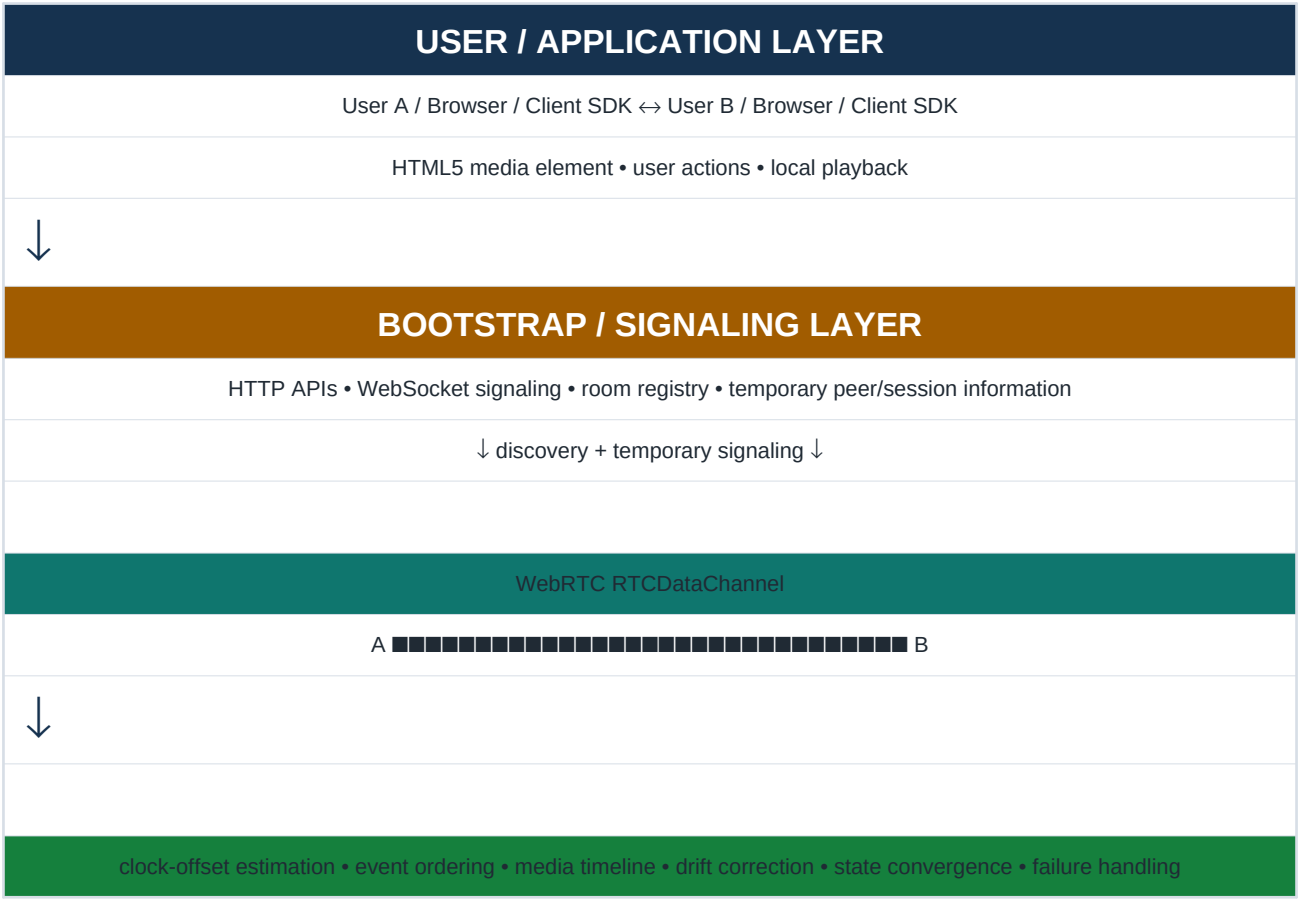
Component	Primary responsibility	Phase 1 status
Browser / Client SDK	User interaction, media playback and participation in peer communication.	Implemented conceptually
Bootstrap Server	Room/session discovery, peer registration and temporary signaling.	Initial infrastructure
WebRTC Layer	Establishes browser-to-browser communication.	Initial transport
Synchronization Protocol	Clock, events, state convergence and drift handling.	Research / next phase
Media Layer	HTML5 media element used for the first synchronization experiment.	Initial experiment

3.4 Why this decomposition matters

The separation prevents the bootstrap service from becoming the hidden central controller. The server owns only the information needed to form the network. The peer layer is deliberately designed as the location for the synchronization intelligence.

4. High-Level System Architecture

Initial architecture from the design, expanded into clear logical layers.


USER / APPLICATION LAYER
User A / Browser / Client SDK ↔ User B / Browser / Client SDK
HTML5 media element • user actions • local playback
↓
BOOTSTRAP / SIGNALING LAYER
HTTP APIs • WebSocket signaling • room registry • temporary peer/session information
↓ discovery + temporary signaling ↓
WebRTC RTCDataChannel
A ■■■

5. Detailed End-to-End Workflow

Room creation → discovery → signaling → WebRTC → direct peer synchronization.

1	**Application startup** User A opens the application. The browser starts the client SDK and media interface.
2	**Create room** The client sends **POST /rooms**. The bootstrap server creates/registers a synchronization room.
3	**Receive identifiers** The server returns a **room/session ID** and a temporary **peer ID** to A.
4	**Share/join room** User B opens the application and provides the room ID to join the same synchronization session.
5	**Register participant** The server registers B as another participant in the room and provides peer/session information.
6	**Temporary WebRTC signaling** A and B exchange WebRTC offer/answer and ICE candidate information through the bootstrap signaling channel.
7	**Establish P2P connection** WebRTC attempts to establish a direct browser-to-browser connection.
8	**Begin peer synchronization** After connection establishment, synchronization messages are exchanged directly between peers.
9	**Bootstrap role becomes inactive** The bootstrap server is removed from ordinary PLAY/PAUSE/SEEK synchronization traffic.

Sequence view

```
A → Bootstrap: CREATE_ROOM
Bootstrap → A: ROOM_ID + PEER_ID
B → Bootstrap: JOIN_ROOM(ROOM_ID)
Bootstrap → A/B: signaling information
A ↔ B: WebRTC connection establishment
A ↔ B: SYNC_PROTOCOL messages
```

Important transition

The architecture has two distinct communication phases: (1) server-assisted connection setup and (2) peer-to-peer synchronization. This transition is the defining feature of the initial system design.

6. Bootstrap Server Design

Minimal API, minimal room state and short-lived server responsibility.

6.1 Proposed API

Endpoint	Purpose	Lifetime / role
POST /rooms	Create a synchronization room.	Room creation
POST /rooms/{roomId}/join	Join an existing room and return peer/session information.	Room discovery
WebSocket /rooms/{roomId}/signal	Exchange WebRTC offer/answer/ICE information.	Temporary signaling
DELETE /rooms/{roomId}/peers/{peerId}	Optional explicit departure notification.	Membership cleanup

6.2 Minimal room data model

```
{
  roomId: "abc123",
  peers: [
    { peerId: "peerA", joinedAt: ... },
    { peerId: "peerB", joinedAt: ... }
  ],
  expiresAt: ...
}
```

6.3 State ownership

Server-owned bootstrap state	Peer-owned synchronization state
Room/session ID	Playback position
Temporary peer/session IDs	PLAY/PAUSE/SEEK state
Participant membership during bootstrap	Clock offset and group timing
Expiry/cleanup information	Drift information and correction decisions
Temporary signaling messages	Future distributed/convergent media state

For the first prototype, an in-memory room registry is sufficient. Persistent storage is not required unless authentication, recovery or multi-instance deployment makes it necessary.

7. Server Lifecycle and Technology Stack

The bootstrap server has a deliberately short-lived role.

7.1 Server lifecycle

START APPLICATION

↓

CREATE / JOIN ROOM

↓

DISCOVER PEERS

↓

EXCHANGE WEBRTC SIGNALING

↓

ESTABLISH P2P CONNECTION

↓

BOOTSTRAP ROLE COMPLETE

↓

PEER-TO-PEER SYNCHRONIZATION

7.2 Initial technology stack

Layer	Technology	Purpose
Client	JavaScript / TypeScript	Browser-based client SDK
Peer communication	WebRTC RTCDataChannel	Direct peer messaging
Bootstrap transport	HTTP + WebSocket	Room API and temporary signaling
Server	Node.js / TypeScript	Lightweight initial service
Media	HTML5 HTMLMediaElement	First media synchronization experiment

7.3 Reconnection

If a peer disconnects, the first implementation may temporarily use the bootstrap server again for reconnection signaling. The server's role remains connectivity assistance; it does not become the synchronization authority.

8. Security, Reliability and Failure Considerations

Minimal infrastructure still needs controlled access and cleanup.

8.1 Initial security controls

Area	Initial design consideration
Room access	Use unguessable room IDs or authenticated room access.
Peer identity	Use short-lived peer/session identifiers.
Signaling validation	Validate signaling messages before forwarding/processing them.
Room cleanup	Expire rooms and remove unused state.
Rate limiting	Apply rate and connection limits to reduce abuse.
Signaling injection	Protect rooms against arbitrary users injecting signaling data.

8.2 Reliability questions for later phases

• What happens when one peer leaves during synchronization?

• How is a new peer added after the initial connection has already formed?

• How are clock differences estimated accurately?

• How is network delay and jitter measured?

• How are simultaneous PLAY/PAUSE/SEEK events ordered deterministically?

• How is playback drift detected and corrected?

• What happens when the network is partitioned and later reconnects?

• How should faulty or malicious peers be handled?

8.3 Important architectural observation

A leaderless system cannot simply assume that one central component will resolve every conflict. Therefore, peer identity, event ordering, state aggregation, convergence and failure handling become central design problems.

9. Evolution from Initial Prototype to Target Architecture

The final topology is intentionally left open for experimental evaluation.

INITIAL	Bootstrap Server → WebRTC → Peer A ↔ Peer B
INTERMEDIATE	Bootstrap Server → WebRTC → Peer Mesh → distributed clock + distributed state
TARGET	Minimal Bootstrap → Peer Network → Leaderless Synchronization → Convergent Media Timeline

9.1 Research layers after bootstrap

01	Peer identity and membership protocol	07	Leaderless state aggregation
02	Clock-offset estimation	08	Drift detection and correction
03	RTT / jitter measurement	09	Peer failure and reconnection handling
04	Distributed media timeline	10	Network partition and merge behavior
05	PLAY / PAUSE / SEEK / RATE protocol	11	Faulty / malicious peer handling
06	Deterministic event ordering	12	Convergence and synchronization-quality measurement

9.2 Choosing the final peer topology

The design deliberately does not commit to a single final topology. Experiments are expected to compare full mesh, partial mesh, gossip-based dissemination or another topology against synchronization accuracy, bandwidth usage, scalability and fault tolerance.

10. Phase 1 Success Criteria and Implementation Roadmap

A clear definition of done makes the first prototype measurable.

10.1 Phase 1 success criteria

✓	Multiple browser clients can create and join the same room.
✓	Peers receive unique session/peer identities.
✓	WebRTC signaling succeeds.
✓	Browsers establish direct peer connections where possible.
✓	Synchronization messages can be exchanged through the peer connection.
✓	The bootstrap server is not required for ordinary PLAY/PAUSE/SEEK synchronization traffic.

10.2 Suggested implementation sequence

1	**Build the room API** Implement room creation, joining, peer registration and expiry.
2	**Add WebSocket signaling** Create the temporary signaling channel for a room.
3	**Integrate WebRTC** Create offer/answer and ICE exchange and establish RTCDataChannel connectivity.
4	**Connect the media layer** Use the HTML5 media element for the first experiment.
5	**Send basic peer messages** Verify that peers can exchange structured synchronization messages directly.
6	**Measure the bootstrap boundary** Confirm that normal synchronization does not depend on the server.
7	**Begin distributed synchronization research** Add clock estimation, event ordering, drift correction and convergence mechanisms in later phases.

Phase 1 deliverable

A working prototype in which users can create/join a room, establish a direct WebRTC connection, and exchange peer-to-peer synchronization messages while the bootstrap server remains limited to setup duties.

11. System Design Quick Reference

Key terms and architectural decisions in one place.

Glossary

Term	Meaning in this project
Bootstrap server	Initial infrastructure used for session discovery, room management and connection setup.
Signaling	Temporary exchange of WebRTC connection information such as offer/answer and ICE candidates.
WebRTC	Browser technology used to establish peer-to-peer communication.
Peer	A participating browser/client in the synchronization session.
Leaderless	Target design where no permanent central leader is responsible for synchronization decisions.
Distributed state	Synchronization-related state maintained and reasoned about across peers.
Drift	Difference in playback timing between participants; correction is a future protocol responsibility.
Convergence	Future goal that participating peers reach a consistent synchronized media timeline.

Architecture in one line

CREATE/JOIN ROOM → DISCOVER PEERS → SIGNALING → WEBRTC → DIRECT P2P CONNECTION → PEER SYNCHRONIZATION

Final design principle

The bootstrap server is an initial infrastructure component, not the synchronization system itself. Its job is to solve the practical browser networking problem. Once peers are connected, the important logic moves toward the distributed peer layer: clock estimation, media timeline synchronization, event ordering, drift correction, state convergence and failure handling.

Document basis: expanded from the supplied “Initial System Design: Bootstrap Server” Phase 1 document. The final architecture remains intentionally open pending scalability and synchronization experiments.